

NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing

Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham[†],
Gwangsun Kim[†], Divya Mahajan[§], Jongse Park

KAIST, [†]POSTECH, [§]Georgia Institute of Technology

Presenter: Sushil Raj Regmi

LLM Inference: Computational Characteristics

Two phases:

- **Summarization Phase:** multiple tokens processed in parallel
- **Generation Phase:** tokens generated sequentially

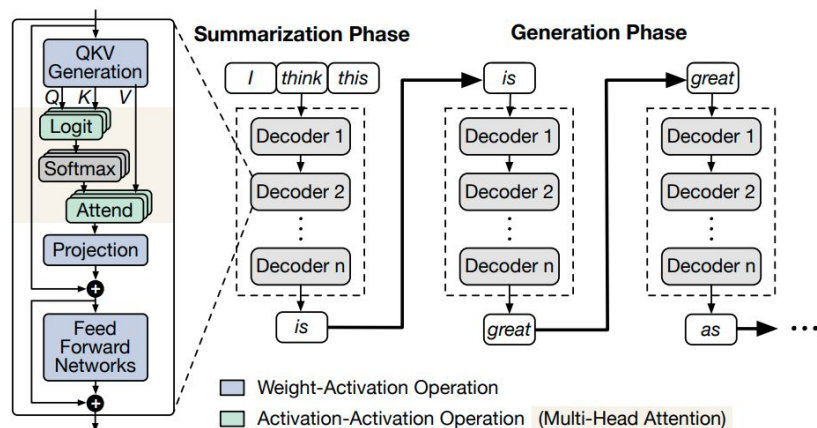
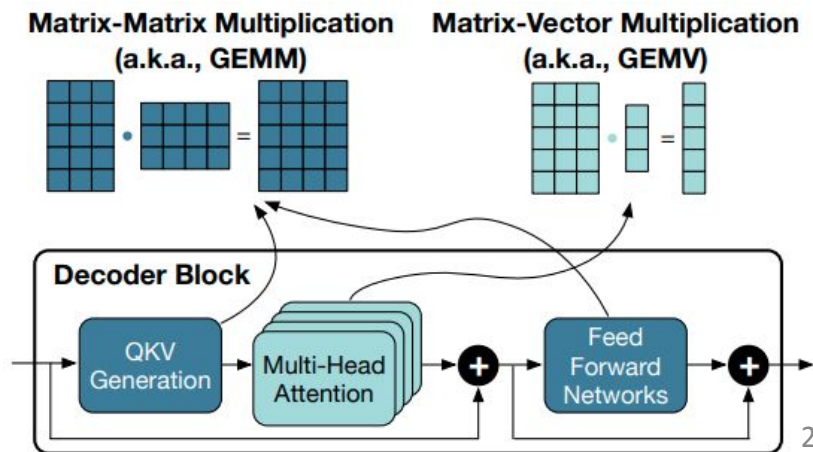


Figure 2. Model architecture and inference in LLMs.

Key Computation:

- **GEMM (Matrix-Matrix):** efficient, high utilization
- **GEMV (Matrix-Vector):** bandwidth-bound, low utilization



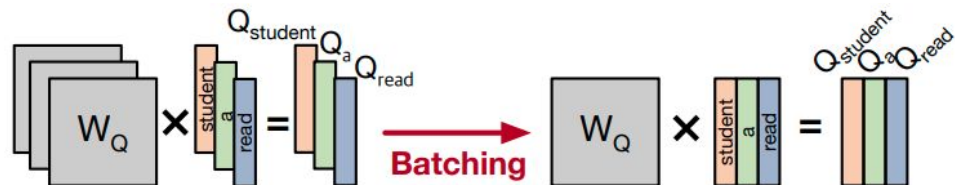
LLM Inference: Batching

Weight-Activation Multiplication (QKV, FFN):

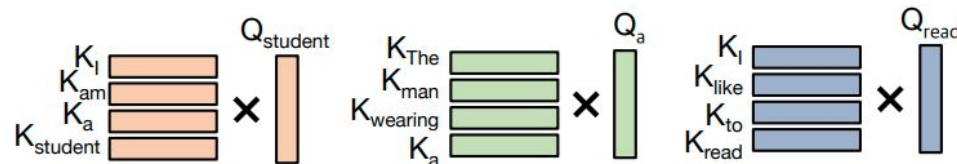
- Normally GEMV operation
- Weights can be shared
- With batching, converted to GEMM

Activation-Activation Multiplication (MHA):

- Current token (vector) * all past tokens (matrix)
- Both query and Key/Value are unique activations per request → no reuse
- Always GEMV (bandwidth bound)



(a) Weight-Activation Operation



(b) Activation-Activation Operation

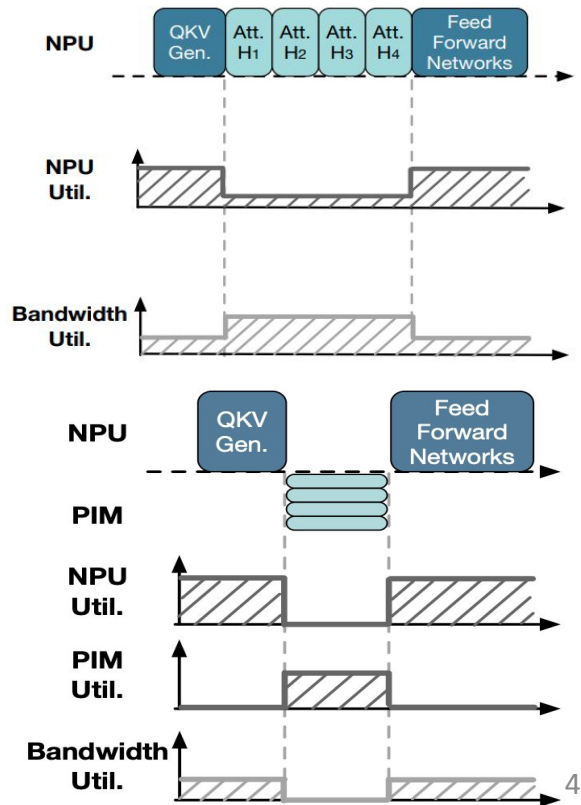
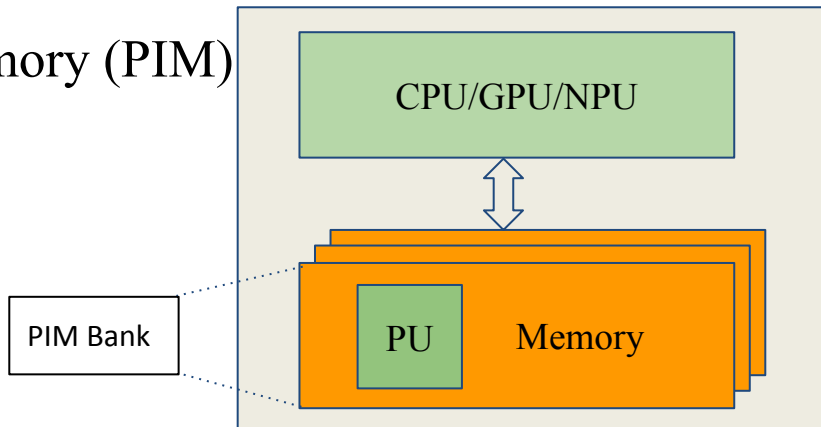
LLM Inference: Compute Under-Utilization

- GPUs and NPUs are inefficient for GEMV operation
- Fluctuating utilization



Offload to GEMV efficient accelerators

Processing In Memory (PIM)



LLM Inference: Compute Under-utilization

- GPU capacity nearly 100%, but compute utilization < 40%
- Bottleneck from GEMV ops in MHA → bandwidth limits prevent full GPU usage.

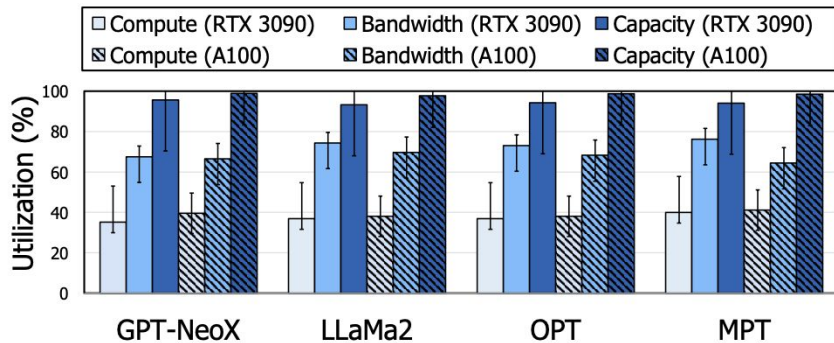


Figure 5. GPU resource utilization for four different LLMs.

- Naive NPU-PIM integration is sequential → NPU works on QKV/FFN while PIM is idle, and PIM runs MHA while NPU is idle.
- Overall utilization < 40%

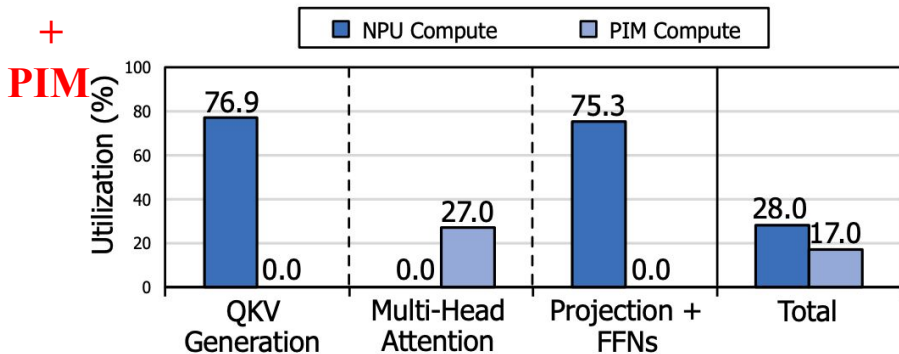
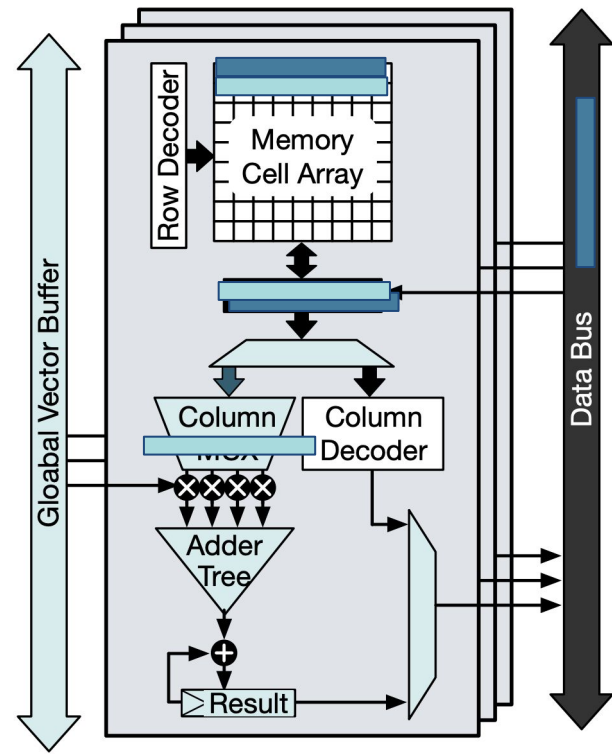
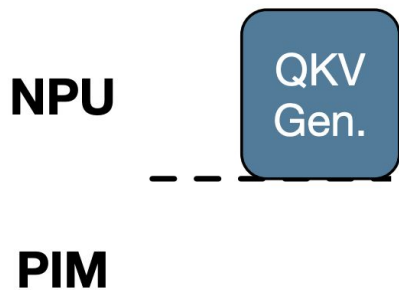


Figure 6. NPU-PIM resource utilization for decoder block.

Necessity of concurrent NPU and PIM executions.

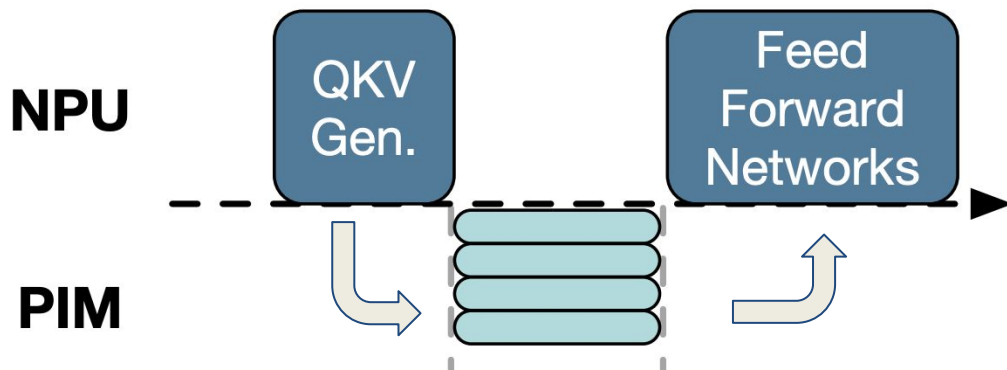
Challenges: Blocked PIM Mode

- Current PIM GEMV accelerators use a single row buffer per memory bank.
- Row buffer handles both:
 - Regular memory read/write
 - PIM computation (GEMV)
- Modes are mutually exclusive

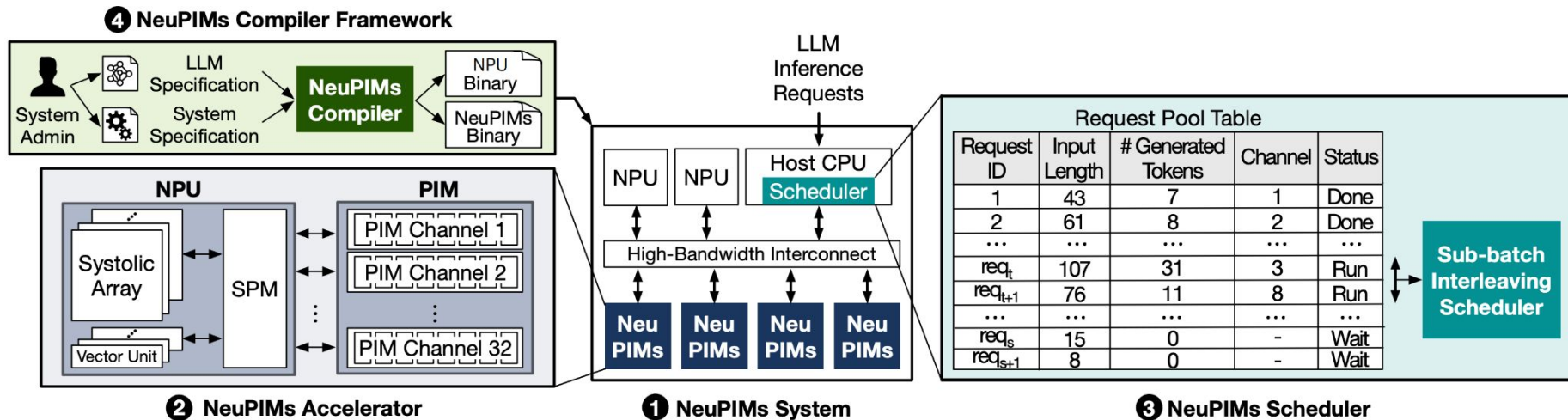


Challenges: GEMM-GEMV Dependency

- Data dependency in decoder blocks: GEMM outputs (QKV projections) are required before GEMV (MHA) can run.
- Forces sequential execution of GEMM (NPU) and GEMV (PIM), limiting opportunities for parallel NPU-PIM computations.



NeuPIMs: Overview



PIM Microarchitecture for Concurrent Execution

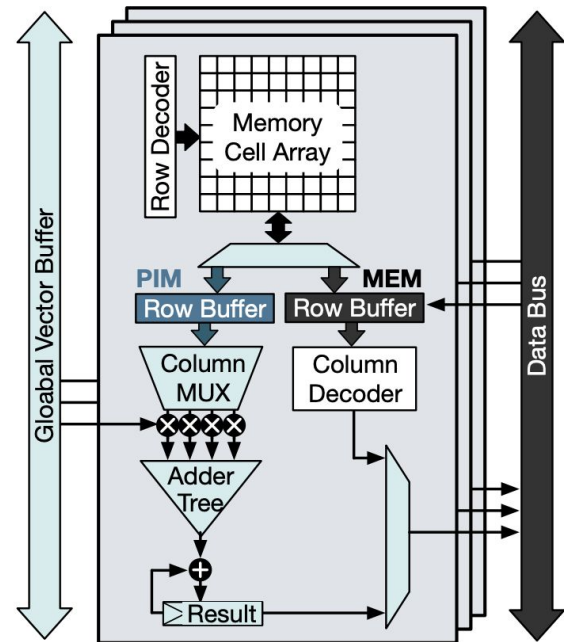
Dual Row Buffer in PIM:

- MEM row buffer → handles regular memory read/write.
- PIM row buffer → dedicated for GEMV operations.

Two independent data paths → memory access and GEMV can run concurrently.

Design principle:

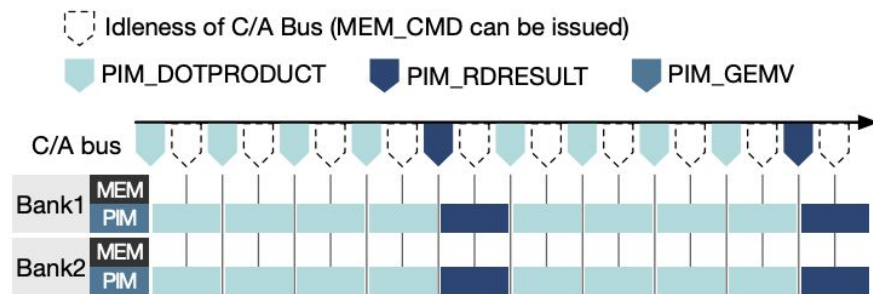
- Minimal hardware modification → avoids large area/power overheads.
- Complexity shifted to command interface + memory controller.



NeuPIMs Memory Command Interface

Baseline PIM Commands:

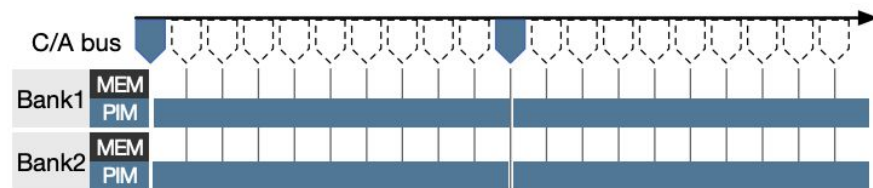
- *PIM_GWRITE* → Copy vector operand to global buffer
- *PIM_ACTIVATION* → Activate matrix rows into PIM row buffers
- *PIM_DOTPRODUCT* → Perform parallel in-memory dot products
- *PIM_RDRESULT* → Send results back to host



(a) Baseline C/A bus idleness

NeuPIMS Extensions:

- *PIM_HEADER* → Communicates GEMV dimensionality; enables latency estimation & safe scheduling with DRAM refresh
- *PIM_GEMV* → Composite command: executes multiple dot-products & returns result at once (reduces command traffic)
- *PIM_PRECHARGE* → Precharge PIM row buffer (needed due to dual row buffers)



(b) NeuPIMS C/A bus idleness

NeuPIMs Scheduling

Overlapping in MHA layer

- Head level parallelism: logit, softmax, attend split across heads
- Naive Integration: results cannot be transferred between PIM and NPU efficiently
- NeuPIMs dual row buffer allow for concurrent usage

Limitation:

- Overlap possible only at head level parallelism
- Systolic arrays remain underutilized during MHA execution

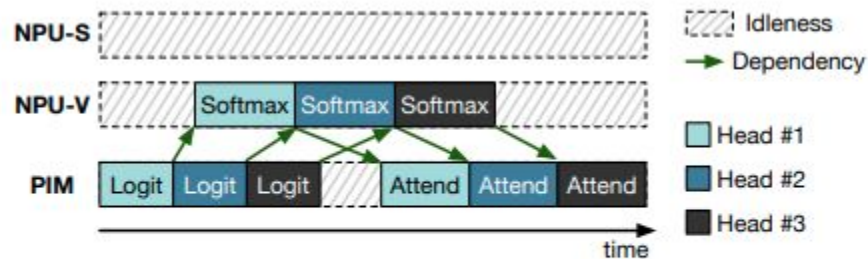
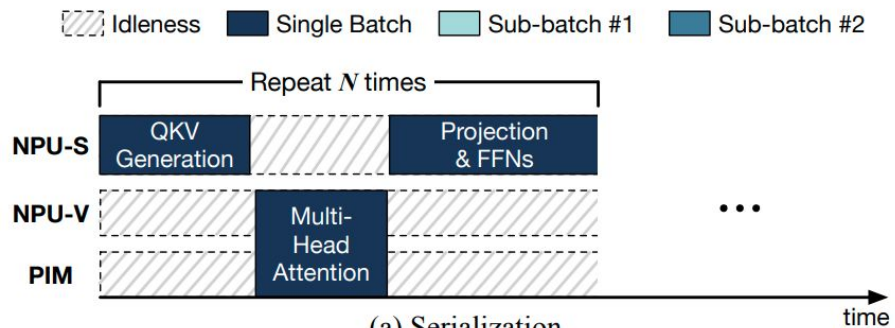


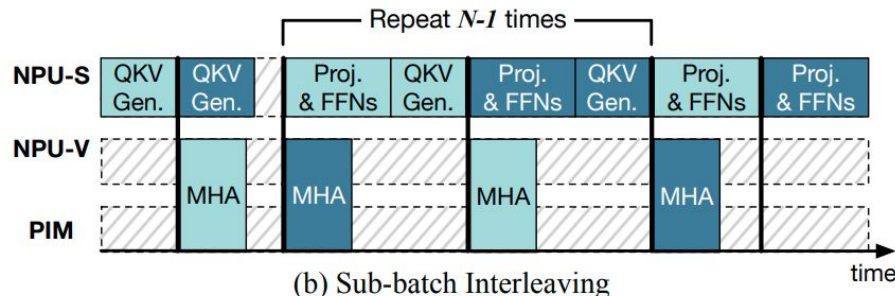
Figure 10. Overlapping opportunities of multi-head attention layers. NPU-S: systolic arrays; NPU-V: vector units.

Sub-Batch Interleaving

- Partition one large batch into two sub-batches
- PIM latency for MHA effectively hidden
- Higher utilization of both NPU and PIM



(a) Serialization



(b) Sub-batch Interleaving

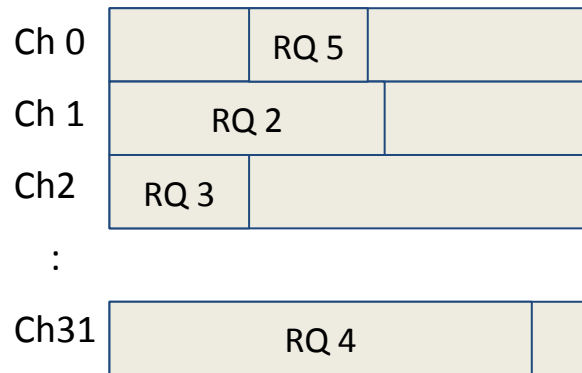
Sub-batch Interleaving: Challenges

Channel load balancing

- MHA latency depends on the slowest channel (longest sequence)
- Uneven token lengths create bottleneck
- Requires load balancing across channels

Subbatches partitioning

- Two sub-batches must finish in similar time.
- Slow sub-batch stalls overall interleaving.
- Needs careful partitioning to balance execution times.



Multi-Head Attention Latency Estimation

Algorithm 1: MHA Latency Estimation

Input: seq_len : Sequence length of the request
Parameter: E : Model embedding size
 L_{tile} : GEMV latency for one PIM tile
 L_{GWRITE} : GWRITE latency for global buffer
 E : Model embedding size
 P_{DRAM} : DRAM page size
 B_{chnl} : PIM banks per channel
 N_{head} : Number of heads
Output: $LMHA$: Estimated latency for MHA

```
1  $LMHA \leftarrow 0$ ;  
   /* GEMV latency for  $Key^T \times Query$  */  
2  $N_{tiles} \leftarrow (seq\_len / B_{chnl}) * (E / P_{DRAM})$ ;  
3  $LMHA += L_{GWRITE} * (E / P_{DRAM})$ ;  
4  $LMHA += L_{tile} * N_{tiles}$ ;  
  
   /* GEMV latency for  $Logits \times Value$  */  
5  $N_{tiles} \leftarrow ((E / N_{head}) / B_{chnl}) * ((seq\_len / P_{DRAM}) * N_{head})$ ;  
6  $LMHA += L_{GWRITE} * ((seq\_len / P_{DRAM}) * N_{head})$ ;  
7  $LMHA += L_{tile} * N_{tiles}$ ;  
8 return  $LMHA$ 
```

The algorithm provides the latency of one MHA layer execution for a single request (sequence), based on its sequence length and embedding size under the NeuPIMs memory layout.

Greedy Min-Load Bin Packing Algorithm

- MHA latency estimation
- Sort requests by decreasing sequence length
- Greedily allocate each request to least loaded channel
- Update channel load after each allocation
- Repeat until all request are assigned.

Algorithm 2: Greedy Min-Load Bin Packing

Input: L_{req} : A list for squence length of new requests
 L_{chnl} : A list for request allocation of channels

/ Calculate each channel's total load by applying MHA latency estimation to each allocated request */*

```
1  $L_{load} \leftarrow []$ ;  
2 foreach  $chnl$  in  $L_{chnl}$  do  
3    $Sum_{load} \leftarrow 0$ ;  
4   foreach  $req$  in  $chnl$  do  
5      $Sum_{load} += \text{MHALatencyEstimation}(req)$   
6   end  
7    $L_{load}.append(Sum_{load})$ ;  
8 end  
  
   /* Allocate each request by greedy algorithm */  
9 foreach  $new\_req$  in  $L_{req}$  do  
10   $min\_index = \text{Min}(L_{load}).index()$ ;  
11   $L_{chnl}[min\_index].append(new\_req)$ ;  
12   $load_{req} = \text{MHALatencyEstimation}(new\_req)$ ;  
13   $L_{load}[min\_index] += load_{req}$ ;  
14 end  
15 return  $L_{chnl}$ 
```

Sub-batch Partitioning Algorithm

- Ensures two sub-batches have similar size for efficient interleaving.
- Splits requests in each channel into two halves.
- If odd number of requests $\rightarrow$ alternates ceil/floor to balance load.

Algorithm 3: Sub-Batch Partitioning

Input: L_{req} : A list of active request set in each channel

Output: SB_1, SB_2 : Sub-batches for interleaving

```
1  $turn \leftarrow True$ ;  
2  $SB_1, SB_2 \leftarrow []$ ;  
3 foreach  $req_{chnl}$  in  $L_{req}$  do  
4    $b_{size} \leftarrow \text{Size}(req_{chnl}) / 2$ ;  
5   if  $\text{Size}(req_{chnl}) \% 2 \neq 0$  then  
6      $b_{size} = turn ? \text{ceil}(b_{size}) : \text{floor}(b_{size})$ ;  
7      $turn = !turn$ ;  
8   end  
9    $b_{size} = \text{int}(b_{size})$ ;  
10   $SB_1.append(req_{chnl}[: b_{size}])$ ;  
11   $SB_2.append(req_{chnl}[b_{size} :])$ ;  
12 end  
13 return  $SB_1, SB_2$ 
```

Scaling NeuPIMs System

NeuPIMs devices have limited memory → require model parallelism for LLM inference.

Pipeline parallelism:

- Split model layer-wise across devices.
- Uses micro-batches for pipelining.
- In NeuPIMs: reduces decoder blocks per device → lowers performance benefits.
- Sub-batch interleaving further reduces batch size → underutilizes NPU systolic arrays.

Tensor Parallelism:

- Split tensors across devices; shards processed in parallel.
- Communication overhead is modest (traffic unchanged).
- Overlap: one sub-batch communicates while other computes → hides latency.

Insight: Prefer tensor parallelism first; use pipeline parallelism only when model is too large.

Evaluation

Simulator

- Cycle-level simulator, ONNXim, DRAMsim

Comparison target

- GPU-only: A100 40 GB
- NPU-only: Accelerator with systolic array unit vector unit
- NPU-PIM: NPU integrated with PIM for GEMV operations

Workload

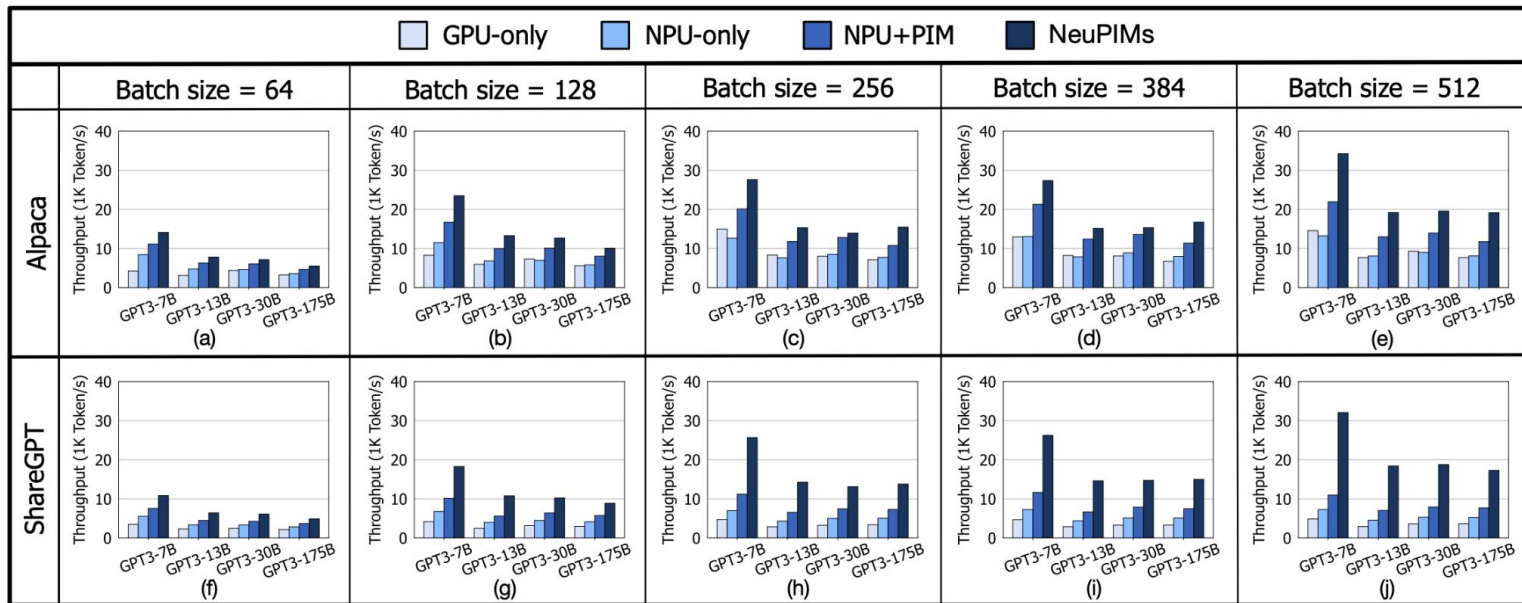
- LLM model: GPT-3
- Dataset: ShareGPT, Alpaca

NPU Configuration		HBM Organization	
Systolic Arrays/Chip	8	Banks/Bankgroup	4
Vector Units/Chip	8	Banks/Channel	32
HBM Channels/Chip	32	Frequency	1GHz
Systolic Array Size	128 x 128	Capacity/Channel	1GB
Vector Unit Size	128 x 1	Page Size	1KB
HBM Timing Parameter			
tRP = 14, tRCD = 14, tRAS = 34, tRRD_L = 6, tWR = 16, tCCD_S = 1, tCCD_L = 2, tREFI = 3900, tRFC = 260, tFAW = 30			

Table 3. The evaluated LLM configurations.

Model	# Layers	# Heads	d_{model}	# TP	# PP
GPT3-7B	32	32	4096	4	1
GPT3-13B	40	40	5120	4	1
GPT3-30B	48	56	7168	4	2
GPT3-175B	96	96	12288	8	4

Evaluation: Throughput (token/s)



- NPU-PIM shows on average $1.5\times$ higher throughput compared to NPU-only.
- NeuPIMs achieve $1.13\times$ – $3\times$ higher throughput compared to NPU-PIM.
- As batch size increases, NeuPIMs can fully utilize the computing resources of NPU+PIM.

Evaluation: Hardware Utilization

Hardware utilization

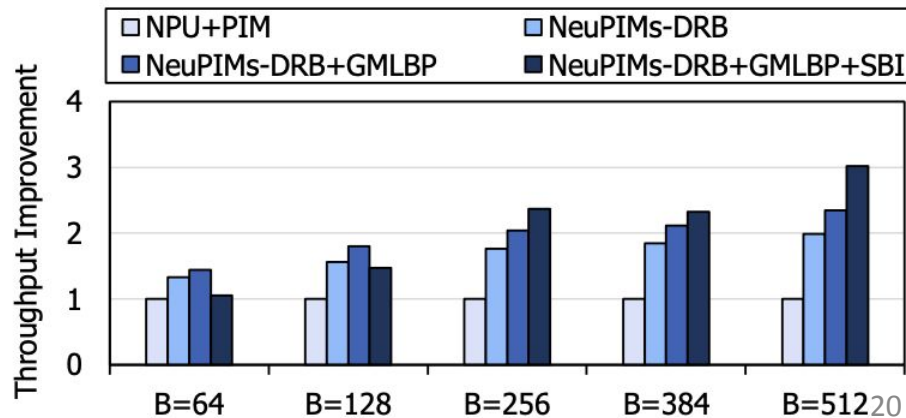
- Due to DRB (Dual Row Buffer) and SBI (Sub-Batch Interleaving), idle states of HW resources are reduced, achieving higher utilization than baseline comparison targets.

Ablation Study:

For small batches, splitting may overwhelm the efficiency of systolic arrays.

Table 4. Average resource utilization of NPU/PIM compute resource and memory bandwidth utilization.

	NPU-only	NPU+PIM	NeuPIMS
NPU	12.3%	28.0%	64.9%
PIM	-	17.0%	26.4%
Bandwidth	67.6%	27.4%	85.4%



Evaluation: Ablation Study

Parallelization schemes

- Scaling NeuPIMs requires both tensor (TP) and pipeline parallelism (PP).
- Tensor parallelism preferred $\rightarrow$ maintains larger batch size $\rightarrow$ better NPU efficiency.
- Across GPT-7B $\rightarrow$ GPT-175B: throughput decreases as per-device batch size shrinks (lower NPU utilization).

Area Overhead

- Extra hardware = dual row buffer.
- Measured via CACTI 7.0 (22nm) $\rightarrow$ 3.11% area overhead.

Power Overhead

- NeuPIMs runs PIM + NPU concurrently
 $\rightarrow$ higher memory power.
- Dual row buffers increase background power.
- Average power:
 - NPU-only: 364.1 mW
 - NeuPIMs: 634.8 mW ($\approx 1.8\times$ higher)

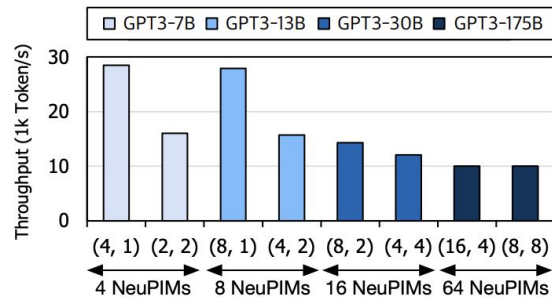


Figure 14. Throughput of multi-NeuPIMs system as the parallelization schemes change. We employ four combinations of tensor parallelism (TP) and pipeline parallelism (PP), represented as (TP, PP).

Table 5. NeuPIMs power overhead.

Baseline	Average Power	
NPU-only	HBM (non-PIM)	364.1 mW
NeuPIMs	Dual row buffered PIM	634.8 mW

Limitations

- NeuPIMs assumes classic MHA where attention is GEMV-dominated. However, newer LLMs (e.g., LLaMA) adopt MQA/GQA, which structure attention to different structure. NeuPIMs may not benefit in such settings, reducing its applicability to state-of-the-art models.
- Although NeuPIMs overlaps MHA execution between PIM and vector units, systolic arrays and underutilized channels/sub-batches still experience idle time, preventing full hardware utilization.
- As model size grows and parallelism increases, per-device batch size shrinks, leading to lower NPU utilization and reduced overall throughput.